
Diplom-Arbeitsplanung

Titel: **SmartSubmit**

Alternativer Titel: **Abgabebox**

Projektbeschreibung: Entwicklung einer webbasierten Plattform zur digitalen Aufgabenabgabe und Bewertung in Bildungseinrichtungen

Teammitglieder: Nataliia Hrytsun, Song Jin, Thi Quynh Anh Nong

Inhaltsverzeichnis

1. Motivation	3
2. Zielsetzung	3
3. Ergebnis.....	3
4. Zeit- und Treffplan (Vorschlag).....	3
5. Struktur für die Diplomarbeit.....	4
6. Arbeitspakete	5
Nataliia – Frontend & UI	5
Song – Backend & Datenbank.....	5
Anh – Funktionen & Infrastruktur	6
7. Programmiersprachen & Frameworks (Methoden)	6
8. Werkzeuge & Tools	7
9. Erwartete Datenbank (mehr dazu im Anhang)	7
10. Erweiterungen für die Zukunft.....	10
11. Anhang	12

1. Motivation

Die Digitalisierung verändert zunehmend den Schulalltag. Lehrkräfte und Schüler:innen arbeiten vermehrt mit digitalen Tools, doch viele bestehende Lösungen sind veraltet oder nicht mehr kompatibel mit modernen Anforderungen. Die bisher genutzte Windows-basierte Abgabebox ist funktional veraltet, technisch eingeschränkt und nicht mehr benutzerfreundlich. Lehrkräfte und Schüler:innen wünschen sich eine moderne Lösung, die intuitiv bedienbar, plattformunabhängig und funktional erweiterbar ist. Unser Ziel ist es, mit SmartSubmit eine zeitgemäße Webplattform zu entwickeln, die den digitalen Schulalltag aktiv unterstützt – durch strukturierte Aufgabenverwaltung, einfache Einreichung, transparente Bewertung und klare Kommunikation.

2. Zielsetzung

SmartSubmit soll:

- Digitale Abgabe von Aufgaben vereinfachen
- Lehrkräften ein übersichtliches Bewertungssystem bieten
- Schüler:innen eine klare Rückmeldung ermöglichen
- Schulbetrieb durch Automatisierung und Rollenlogik entlasten

3. Ergebnis

Eine voll funktionsfähige Webanwendung mit 25-30 logisch aufgebauten Datenbanktabellen, rollenbasiertem Zugriff, Datei-Upload, Bewertungssystem, Benachrichtigungen und Statistikmodul – bereit für den Einsatz im Schulalltag und offen für zukünftige Erweiterungen

4. Zeit- und Treffplan (Vorschlag und dynamisch nach dem Verlauf)

Phase	Zeitraum	Inhalt / Ziel
-------	----------	---------------

Kickoff & Planung	KW 38–39	Rollenverteilung, Tools einrichten, Pflichtenheft starten, ER-Diagramm
Datenmodell & Architektur	KW 40–41	API-Struktur (Funktionen Input-Format, Output-Format), Tech-Stack finalisieren
UI-Design & Mockups	KW 42–43	Figma-Designs, Feedbackschleife mit Lehrern
Backend-Entwicklung	KW 44–46	Datenbank, API, Authentifizierung, Datei-Upload
Frontend-Entwicklung	KW 47–49	Dashboard, Formulare, Feedback-Komponenten
Infrastruktur & CI/CD	KW 50–52	Docker, GitHub Actions, Deployment-Test
Tests & Validierung	KW 01–04 (2026)	Unit-Tests, Benutzerfeedback, Bugfixes
Dokumentation & Präsentation	KW 05–16 (2026)	Diplomarbeit schreiben, Präsentation vorbereiten

Treffen: Wöchentliches Teammeeting + 1x-2x/Monat mit Betreuer

5. Struktur für die Diplomarbeit

Abschnitt	Inhalt
Einleitung	Problemstellung, Zielsetzung, Motivation
Stand der Technik	Vergleich mit existierenden Lösungen (z. B. alte Windows-App - die als Referenzsystem zur Analyse und Weiterentwicklung dient)
Anforderungsanalyse	Funktionale & nicht-funktionale (Leistungsanforderungen)
Datenmodell	ER-Diagramm, Tabellenbeschreibung
Architektur & Technologien	Tech-Stack, Frameworks, API-Struktur

Implementierung	Code-Struktur, Module, UI-Design, Algorithmen. Die Benutzeroberfläche soll intuitiv, barrierefrei und responsiv sein. Wir orientieren uns an modernen UI-Prinzipien wie Klarheit, Konsistenz und Feedback.
Tests & Validierung	Testfälle, Benutzerfeedback, CI/CD: Unit-Tests für Backend-Logik Integrationstests für API-Endpunkte Usability-Tests mit echten Schüler:innen & Lehrer:innen CI/CD-Tests über GitHub Actions
Ausblick & Erweiterungen	Zukunftsideen, Skalierung, Open Source
Fazit	Reflexion, Lernerfahrung, Nutzen für Schule

6. Arbeitspakete

Nataliia – Frontend & UI

- **React-Komponenten**
 - + Aufbau modularer UI-Komponenten (z. B. Navigationsleiste, Upload-Formular, Bewertungsansicht)
 - + Wiederverwendbare Strukturen für Aufgabenlisten, Einreichungen und Feedback
- **Tailwind CSS Styling**
 - + Anwendung eines konsistenten Designsystems mit Tailwind
 - + Responsive Layouts für Desktop, Tablet und Mobile
 - + Farb- und Typografie-Konzept passend zum Branding von „SmartSubmit“
- **Dashboard-Entwicklung**
 - + Schüler-Dashboard: Übersicht über offene Aufgaben, Abgabefristen, Status

-
- + Lehrer-Dashboard: Einreichungsstatus, Bewertungsübersicht, Filterfunktionen
 - **Formulare & Interaktion**
 - + Upload-Formular mit Drag & Drop, Dateivorschau
 - + Bewertungsformular für Lehrer mit Noten, Kommentaren und Statusauswahl
 - + Validierung von Eingaben (z. B. Pflichtfelder, Dateiformat)
 - + API-Integration
 - + Kommunikation mit dem Backend über REST-API
 - + Authentifizierung, Datenabruf, Datei-Upload, Feedback senden
 - **Usability & Testing**
 - + Durchführung von Usability-Tests mit echten Nutzer:innen
 - + Optimierung der Benutzerführung und Barrierefreiheit

Song – Backend & Datenbank

Die Arbeit umfasst die zentralen Backend-Funktionen des Projekts, die mit Node.js und Express umgesetzt werden:

- Node.js & Express, Datenmodell, REST-API
 - + **Node.js & Express:** Aufbau und Konfiguration des gesamten Node.js-Projekts. Dies beinhaltet die Einrichtung der Entwicklungsumgebung, die Installation des Express-Frameworks und die Konfiguration des Servers.
 - + **Datenmodell:** Entwurf und Definition der Datenbanktabellen (z. B. für Benutzer, Aufgaben, Bewertungen), DIE Zusammenarbeit von alle drei wird empfohlen. Eine Bibliothek wie **mysql2** von Node.js oder ein **ORM** wie **Prisma** verwenden, um die Verbindung zu einer MySQL-Datenbank herzustellen und diese zu verwalten.
 - + **REST-API:** Nutzung der Express-Routing-Funktionen, um alle API-Schnittstellen zu erstellen. Diese Schnittstellen verarbeiten

verschiedene Anfragen vom Frontend, wie Benutzeranmeldungen, Datei-Uploads und Datenabfragen.

- Authentifizierung, Rollenlogik
 - + **Authentifizierung:** Implementierung der Backend-Logik für Benutzeranmeldung, Registrierung und Verifizierung, um sicherzustellen, dass nur legitime Benutzer auf das System zugreifen können. Es ist empfohlen, **bcrypt** zur sicheren Speicherung von Passwörtern und **jsonwebtoken (JWT)** zur Verwaltung von Benutzersitzungen und zur Authentifizierung zu verwenden. Sie gehören zum Ökosystem von Node.js, sind aber keine integrierten Bestandteile von Node.js oder Express. Sie müssen über npm (Node Package Manager) installiert und können dann verwendet werden.
 - + **Rollenlogik:** Programmierung der Logik zur Definition und Durchsetzung von Berechtigungen für verschiedene Benutzerrollen (z. B. Student, Lehrer, Administrator). Zum Beispiel werden Sie Middleware erstellen, um den Zugriff auf bewertungsbezogene API-Endpunkte auf Lehrer zu beschränken.
- **Datei-Upload, Bewertungssystem**
 - + **Datei-Upload:** Einsatz von **Express-Middleware** wie **multer**, um den Empfang, die Verarbeitung und die sichere Speicherung von eingereichten Aufgabendateien zu implementieren.
 - + **Bewertungssystem:** Programmierung der Geschäftslogik, um Bewertungen und Feedback von Lehrern für eingereichte Aufgaben zu verarbeiten und diese Daten in der Datenbank zu speichern.

Anh – Funktionen & Infrastruktur

- **Benachrichtigungssystem**
 - + Eventbasierte Trigger (z. B. „Neue Aufgabe verfügbar“, „Feedback erhalten“)
 - + Aufbau eines Nachrichtencenters für Schüler:innen und Lehrer:innen

-
- + Option für E-Mail-Benachrichtigungen oder In-App-Messages
 - **Cronjobs & Automatisierung**
 - + Regelmäßige Überprüfung von Deadlines
 - + Erinnerungen vor Abgabefristen
 - + Archivierung von Aufgaben nach Semesterende
 - **Statistikmodul**
 - + Berechnung von Abgabequoten, Durchschnittsnoten, Bewertungsverteilung
 - + Visualisierung im Lehrer-Dashboard (z. B. Diagramme, Tabellen)
 - + Exportfunktion als PDF oder CSV
 - **PDF-Export**
 - + Generierung von PDF-Dokumenten für Einreichungsübersichten, Bewertungen
 - + Formatierung mit Logo, Metadaten und Layout-Vorlagen
 - **Docker-Setup**
 - + Containerisierung der Anwendung für einfache Installation und Deployment
 - + Erstellung von Dockerfiles für Backend, Frontend und Datenbank
 - **CI/CD mit GitHub Actions**
 - + Automatisierte Tests bei jedem Commit
 - + Deployment auf Testserver oder Schulserver
 - + Monitoring von Build-Status und Fehlern
 - **Systemereignisse & Logging**
 - + Protokollierung von Aktionen (z. B. Datei gelöscht, Bewertung geändert)
 - + Sicherheitslog für Admins (Login-Zeiten, IP-Adressen)

7. Programmiersprachen & Frameworks (Methoden)

Bereich	Methode	Warum?
Backend	Node.js + Express	Leichtgewichtig, schnell bei I/O-Operationen und mit einem großen Ökosystem von Paketen, ideal für ein JavaScript-Team.
Frontend	React + Tailwind CSS (alternativ: Vue.js)	Modernes UI, komponentenbasiert, responsiv und leicht erweiterbar.
Datenbank	MySQL	Weit verbreitet, einfach einzurichten und ein Standard in vielen Webanwendungen. Arbeitet gut mit Bibliotheken wie mysql2 oder ORMs wie Prisma zusammen.
API	Express	Minimalistisches und flexibles Framework, das eine freie Gestaltung der API-Struktur durch eigene Middleware und Routing ermöglicht.

8. Werkzeuge & Tools

- **GitHub** – Versionskontrolle, Zusammenarbeit, Issues
- **Trello / Notion** – Aufgabenplanung, Meilensteine
- **Figma** – UI-Design & Mockups
- **Postman** – API-Tests
- **Docker** – Containerisierung für Deployment
- **dbdiagram.io** – Datenmodell visualisieren (alternativ: DBeaver)
- **Django Admin** – schnelles Backend-Interface für Lehrer/Admins

9. Erwartete Datenbank (mehr dazu im Anhang)

Die Tabellen sind in folgende Bereiche gegliedert:

- Benutzer & Rollen
- Aufgaben & Einreichungen
- Bewertung & Feedback

- Dateien & Versionierung
- Kommunikation & Benachrichtigungen
- Statistik & Logging
- System & Einstellungen

9. 1. Benutzer & Rollen

Tabelle	Zweck
Benutzer	Zentrale Tabelle für alle Nutzer (Schüler, Lehrer, Admins). Enthält persönliche Daten und Verweise auf Rolle und Klasse.
Rolle	Definiert die Benutzerrolle (z. B. Schüler, Lehrer, Admin) – wichtig für Zugriffskontrolle und UI-Anpassung.
Klasse & Subgruppen	Repräsentiert Schulklassen (z. B. 3AKIFT). Wird Schülern zugeordnet und hilft bei Filterung und Gruppierung.

9.2. Aufgaben & Einreichungen

Tabelle	Zweck
Fach	Liste aller Unterrichtsfächer – wird Aufgaben und Lehrern zugeordnet.
Aufgabe	Enthält Aufgaben, die Lehrer erstellen – inkl. Beschreibung, Fach, Abgabefrist.
AufgabenTyp	Diese Tabelle definiert die Art der Aufgabe, z. B. „Hausübung“, „Projekt“, „Test“, „Prüfung“. Sie hilft dabei, Aufgaben systematisch zu klassifizieren und erleichtert die Filterung, Darstellung und statistische Auswertung. Lehrer wählen beim Erstellen einer Aufgabe den passenden Typ aus.
Tag	Ermöglicht Kategorisierung von Aufgaben (z. B. „Projekt“, „Test“) – für Filter und visuelle Darstellung.
Einreichung	Verknüpft Schüler, Aufgabe und Datei – dokumentiert den Abgabeprozess.
Abgabeversuch	Versionierung von Einreichungen – z. B. bei mehreren Versuchen oder Korrekturen.
Datei	Speichert Metadaten zu hochgeladenen Dateien – inkl. Typ, Größe, Pfad.

Tabelle	Zweck
Dateityp	Liste erlaubter Dateiformate – z. B. PDF, DOCX, ZIP.
Status	Status der Einreichung – z. B. „Eingereicht“, „Bewertet“, „Zurückgewiesen“.

9.3. Bewertung & Feedback

Tabelle	Zweck
Bewertung	Enthält Punkte, Note und Kommentar zur Einreichung – wird vom Lehrer vergeben.
Bewertungsskala	Definiert Bewertungssysteme – z. B. 0–100 Punkte oder Noten 1–5.
Feedback	Lehrerkommentare zur Einreichung – kann auch ohne Bewertung erfolgen.
Kommentarverlauf	Historie aller Kommentare – inkl. Antworten von Schülern oder Folgekommentare.

9.4. Kommunikation & Benachrichtigungen

Tabelle	Zweck
Benachrichtigung	Systeminterne Nachrichten – z. B. neue Aufgabe, Feedback erhalten.
BenutzerEinstellungen	Personalisierung – z. B. Sprache, Dark Mode, Benachrichtigungsoptionen.

9.5. Statistik & Logging

Tabelle	Zweck
Statistik	Aggregierte Daten – z. B. Abgabequote, Durchschnittsnote pro Fach/Klasse.
LoginLog	Sicherheitslog – dokumentiert Logins mit Zeit, IP und Gerät.
SystemEreignis	Logging von Aktionen – z. B. Aufgabe erstellt, Datei gelöscht.

9.6. System & Organisation

Tabelle	Zweck
Archiv	Archivierte Aufgaben nach Semesterende – zur Trennung von aktiven und alten Daten.
Semester	Zeitliche Gliederung des Schuljahres – wichtig für Archivierung und Filterung.
Aufgabe_Semester	Verknüpft Aufgaben mit Semestern – ermöglicht Mehrfachzuordnung.
Lehrer_Fach	Verknüpft Lehrer mit ihren Fächern – für Filterung und Rechtevergabe.

10. Erweiterungen für die Zukunft

10.1. Sicherheit & Authentifizierung

- Zwei-Faktor-Login
- OAuth-Integration (z. B. mit Microsoft oder Google)

10.2. Dateiverwaltung

- Vorschau von PDFs oder Bildern direkt im Browser
- Virensan bei Upload

10.3. KI-Integration

- Automatische Plagiatsprüfung
- Vorschläge für Feedback basierend auf Inhalt

10.4. Lehrer-Dashboard

- Übersicht über alle Einreichungen, Bewertungen, Deadlines (auf Bulme-Moddle)
- Export als Excel/PDF

10.5. Mobile App

- Native App für Android/iOS mit Push-Benachrichtigungen

10.6. Prüfungsverwaltung

- Modul für mündliche Prüfungen inkl. Zeitplan, Raumvergabe, Prüferzuordnung

10.7. Mehrsprachigkeit

- UI in mehreren Sprachen – z. B. Deutsch, Englisch, Chinesisch

10.8. Lehrerkommunikation

- Chatmodul zwischen Schülern und Lehrern
- Gruppennachrichten

10.9. Test- und Übungsmodul

- Interaktive Aufgaben mit automatischer Bewertung (z. B. Multiple Choice)

10.10. Admin-Panel

- Benutzerverwaltung, Rechtevergabe, Systemlogs

11. Anhang



